LINUX INTERVIEW PREPARATION GUIDE

CHAPTER

17

Linux in AWS and Cloud Environments

Interview-focused notes for DevOps, SysAdmin and Cloud Engineers (1–5 years of experience)

IN THIS CHAPTER

01 Introduction
02 Linux on AWS EC2
03 SSH Access to EC2
04 Session Manager
05 User Data and cloud-init
06 Metadata and IMDSv2
07 EBS Volumes
08 Amazon EFS
09 AWS CLI and IAM Roles
10 CloudWatch Agent
11 EC2 Networking
12 Production Scenarios
13 Key AWS Services
14 Interview Q&A
15 Takeaways and Exercises

Ravi Tomar

Linux · DevOps · Cloud Infrastructure

CHAPTER OBJECTIVE

Learn how to administer Linux servers on AWS: accessing EC2 with SSH and Session Manager, automating setup with User Data and cloud-init, using instance metadata, managing EBS and EFS storage, using the AWS CLI with IAM roles, collecting OS metrics with the CloudWatch Agent, and troubleshooting problems that span Linux and AWS.

1 Introduction

Most cloud servers run Linux, and AWS is one of the most common platforms. DevOps engineers launch EC2 instances, deploy applications, configure web servers, monitor resources and troubleshoot production issues there.

On AWS, a problem can come from Linux itself or from the AWS infrastructure around it: IAM, security groups, routing, storage attachments or service endpoints. You need to understand both sides.

2 Linux on AWS EC2

Amazon EC2 provides virtual servers. Common Linux AMIs include Amazon Linux, Ubuntu Server, RHEL, SUSE and Debian. Once running, an instance is administered like any other Linux server.

Common Administration Tasks

1. Connect to the instance and configure access.
2. Install packages and configure web or application services.
3. Attach and mount storage.
4. Configure firewalls, monitoring and backups.
5. Troubleshoot connectivity and application issues.

First Commands After Connecting

```
cat /etc/os-release     # distribution and version
uname -r                # kernel
uptime                  # load and uptime
free -h                 # memory
df -h                   # disk
lsblk                   # attached volumes
ip addr; ip route       # network
```

3 Connecting to EC2 Using SSH

When launching an instance, you choose a key pair. AWS places the public key in the default user's `~/.ssh/authorized_keys`, and you keep the private key (`.pem`). AWS does not let you download the same private key again.

```
# Restrict the private key, or SSH refuses to use it
chmod 400 my-ec2-key.pem

# Ubuntu
ssh -i my-ec2-key.pem ubuntu@PUBLIC_IP

# Amazon Linux / RHEL
ssh -i my-ec2-key.pem ec2-user@PUBLIC_IP
```

Component	Purpose
Private key	Kept by the client to authenticate
Public key	Stored on the server in `authorized_keys`
Security group	Must allow TCP 22 from your IP
Network ACL	Must allow port 22 in and ephemeral ports back out

SSH client configuration, bastion hosts and key debugging are covered in Chapter 7. SSH hardening is covered in Chapter 9.

> **PRODUCTION BEST PRACTICE**
>
> Never share private keys, avoid root login, restrict port 22 to trusted IPs, give each administrator an individual account, and prefer Session Manager to reduce SSH exposure.

4 AWS Systems Manager Session Manager

Session Manager gives you a shell on an instance **without opening port 22 or distributing SSH keys**. Access is controlled through IAM and sessions can be logged.

```
Administrator → AWS Systems Manager → SSM Agent on EC2 (outbound HTTPS) → Linux shell
```

Requirements

- **SSM Agent** running on the instance (preinstalled on Amazon Linux and Ubuntu AMIs).

- **Instance profile** with the `AmazonSSMManagedInstanceCore` policy.
- **Outbound HTTPS** to Systems Manager: through a NAT Gateway, or through VPC endpoints for `ssm`, `ssmmessages` and `ec2messages` in private subnets.
- IAM permission for the administrator to start sessions.

```
# Amazon Linux and RHEL
sudo systemctl status amazon-ssm-agent

# Ubuntu installs the agent as a snap
sudo snap services amazon-ssm-agent

# From an administrator's machine (needs the Session Manager plugin)
aws ssm start-session --target i-0123456789abcdef0
```

INTERVIEW POINT

Session Manager removes the need for **inbound** access, not all network requirements. The agent connects **outbound** to Systems Manager, so it still needs a working agent, an IAM role and a network path to the SSM endpoints.

5 EC2 User Data and cloud-init

User Data provides startup instructions to an instance at launch. On most Linux AMIs, **cloud-init** reads and runs it. A shell script in User Data runs **as root**, and by default only **once**, on the first boot.

Example: Ubuntu

```
#!/bin/bash
apt-get update -y
apt-get install -y nginx
systemctl enable --now nginx
echo "Welcome to my EC2 server" > /var/www/html/index.html
```

Example: Amazon Linux 2023

```
#!/bin/bash
dnf install -y nginx
systemctl enable --now nginx
```

Amazon Linux 2 uses `yum` instead of `dnf`.

Checking What Happened

```
# Wait for cloud-init to finish and show the result
cloud-init status --wait

# Output of your User Data script (the first place to look)
sudo cat /var/log/cloud-init-output.log

# Detailed cloud-init log
sudo less /var/log/cloud-init.log
```

INTERVIEW TRAP

Editing User Data on a running instance does nothing to that instance, because the script has already run on first boot. The new script affects only newly launched instances, unless cloud-init is deliberately configured to run it again.

WARNING

Never put passwords, private keys or access keys in User Data. Anyone with access to the instance can read it from the metadata service at `/latest/user-data`. Use IAM roles and Secrets Manager or Parameter Store instead.

DEVOPS USE CASE

A launch template's User Data installs packages, configures the monitoring agent, creates directories and starts the application service on every new instance. For complex setups, pre-built images (golden AMIs) or configuration management are more reliable than a long User Data script.

6 Instance Metadata and IMDSv2

An instance can read information about itself from `http://169.254.169.254`: instance ID, type, Availability Zone, private IP and temporary IAM role credentials. Chapter 7 introduced the IMDSv2 token flow.

```
# 1. Get a session token (valid up to 6 hours)
TOKEN=$(curl -s -X PUT "http://169.254.169.254/latest/api/token" \
  -H "X-aws-ec2-metadata-token-ttl-seconds: 21600")

# 2. Use it for every request
curl -s -H "X-aws-ec2-metadata-token: $TOKEN" http://169.254.169.254/latest/meta-data/ins
curl -s -H "X-aws-ec2-metadata-token: $TOKEN" http://169.254.169.254/latest/meta-data/ins
curl -s -H "X-aws-ec2-metadata-token: $TOKEN" http://169.254.169.254/latest/meta-data/pla
```

Why IMDSv2 Matters

IMDSv2 requires a token obtained with an HTTP `PUT`. This blocks most SSRF attacks, where an attacker tricks a web application into fetching the metadata URL and stealing the instance's IAM credentials.

- Require IMDSv2 on all instances.
- Use least-privilege IAM roles.
- Never expose metadata responses to application users or logs.

> **NOTE**
> If containers on the instance cannot reach the metadata service, check the IMDS **hop limit**. A limit of 1 allows only the host itself; containers behind a Docker bridge usually need 2.

7 EBS Volumes and Linux Storage

An EBS volume appears to Linux as a block device. Partitioning, filesystems, fstab and growing EBS volumes were covered in detail in Chapter 6; this is the EC2 workflow.

```
# 1. Identify the new volume (FSTYPE empty = no filesystem yet)
lsblk -f

# On Nitro instances, the SERIAL column shows the EBS volume ID
lsblk -o NAME,SIZE,SERIAL

# 2. Create a filesystem (new, empty volumes only)
sudo mkfs.ext4 /dev/nvme1n1

# 3. Mount it
sudo mkdir -p /data
sudo mount /dev/nvme1n1 /data

# 4. Make it persistent using the UUID
sudo blkid /dev/nvme1n1
```

```
UUID=abcd-1234  /data  ext4  defaults,nofail  0  2
```

```
sudo findmnt --verify
sudo mount -a
findmnt /data
```

> **WARNING**
>
> `mkfs` destroys any data on the device. Match the EBS volume ID shown by `lsblk -o NAME,SERIAL` with the AWS console before formatting. Always use `nofail` for extra volumes, so a missing volume cannot stop the server from booting.

Expanding an EBS Volume

1. Increase the volume size in AWS.
2. Confirm Linux sees the new size with `lsblk`.
3. If there is a partition, grow it: `sudo growpart /dev/nvme1n1 1`.
4. Grow the filesystem: `sudo resize2fs /dev/nvme1n1p1` (ext4) or `sudo xfs_growfs /data` (XFS).

If the filesystem sits directly on the disk with no partition, skip `growpart` and run `resize2fs /dev/nvme1n1`. See Chapter 6 for LVM.

EBS Snapshots

Snapshots are point-in-time backups of a volume, used for backup, disaster recovery, safety before risky changes, and creating new volumes from existing data.

> **IMPORTANT**
>
> A snapshot of a running database volume is only crash-consistent. For databases, flush or pause writes, or use the database's own backup tools, to get an application-consistent backup.

8 Amazon EFS and Linux

EFS is managed, shared file storage that many instances can mount at the same time over NFS. It is used for shared uploads, shared web content and files needed by multiple servers.

```
# Option 1: EFS mount helper (amazon-efs-utils, supports TLS)
sudo dnf install -y amazon-efs-utils          # Amazon Linux
sudo mkdir -p /mnt/efs
sudo mount -t efs -o tls fs-1234567890abcdef0:/ /mnt/efs

# Option 2: standard NFS client (works on any distribution)
sudo mount -t nfs4 -o nfsvers=4.1,hard,timeo=600,retrans=2,noresvport \
  fs-1234567890abcdef0.efs.us-east-1.amazonaws.com:/ /mnt/efs

findmnt /mnt/efs
```

For a persistent mount, add `_netdev` to the fstab options so the system waits for the network before mounting:

```
fs-1234567890abcdef0:/  /mnt/efs  efs  _netdev,tls,nofail  0  0
```

EFS Requirements

- A mount target in each Availability Zone where instances run.
- Security groups allowing **NFS on TCP port 2049** from the instances.
- Linux file ownership and permissions still apply; EFS access points can enforce a fixed UID/GID.

Feature	EBS	EFS
Type	Block storage	Shared file storage (NFS)
Access	Normally one instance, one Availability Zone	Many instances across Availability Zones
Size	Fixed; you grow it manually	Grows and shrinks automatically
Typical use	OS disks, databases	Shared application files

9 AWS CLI and IAM Roles

The AWS CLI lets you manage AWS from the Linux terminal for automation, deployments and administration.

```
# Version, and which identity the CLI is using
aws --version
aws sts get-caller-identity

# Common commands
aws ec2 describe-instances
aws s3 ls
aws s3 cp backup.tar.gz s3://my-backup-bucket/
aws s3 cp s3://my-backup-bucket/backup.tar.gz .
```

Use `jq` or `--query` to extract values from CLI output (see Chapter 11).

Use IAM Roles, Not Access Keys

Attach an IAM role to the instance through an **instance profile**. The CLI and AWS SDKs automatically get temporary credentials from the metadata service, which rotate automatically.

For example, a backup script that uploads to one S3 bucket needs a role that allows `s3:PutObject` on that bucket only, and no keys stored anywhere.

WARNING

Never hardcode credentials in scripts, configuration files or Git repositories:

```
AWS_ACCESS_KEY_ID="AKIA..."          # never do this
AWS_SECRET_ACCESS_KEY="..."
```

INTERVIEW TRAP

The CLI checks credentials in order: **environment variables first**, then `~/.aws/credentials`, and the instance role last. Old keys left in an environment variable or credentials file silently override the role and can cause unexpected `AccessDenied` errors. Run `aws sts get-caller-identity` to see which identity is actually in use.

10 CloudWatch Monitoring with the CloudWatch Agent

By default, EC2 reports CPU, network and disk I/O metrics to CloudWatch, but **not memory or disk-space usage**, because those are visible only inside the OS (see Chapter 12). The CloudWatch Agent collects them, plus log files.

Setup

1. Attach an IAM role with the `CloudWatchAgentServerPolicy` policy.

2. Install the agent: `sudo dnf install -y amazon-cloudwatch-agent` on Amazon Linux; other distributions use the package from AWS.
3. Create a configuration file.
4. Load the configuration and start the agent.

```
{
  "metrics": {
    "metrics_collected": {
      "mem":  { "measurement": ["mem_used_percent"] },
      "disk": { "measurement": ["used_percent"], "resources": ["/"] }
    }
  }
}
```

```
# Load the configuration and start the agent
sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl \
  -a fetch-config -m ec2 -s \
  -c file:/opt/aws/amazon-cloudwatch-agent/etc/config.json

sudo systemctl status amazon-cloudwatch-agent
```

> **IMPORTANT**
>
> Starting the service alone is not enough; the agent needs a configuration loaded with `fetch-config`. If no metrics appear, check the agent log at `/opt/aws/amazon-cloudwatch-agent/logs/amazon-cloudwatch-agent.log` and the instance's IAM role.

Useful Alarms

- CPU, memory or disk usage above a threshold.
- Instance status check failures.
- Application or service failures.
- Abnormal network traffic.

For example, an alarm at 80% disk usage gives the team time to act before the filesystem fills up.

11 EC2 Networking and Linux Troubleshooting

An instance can be healthy but unreachable. VPC components, Security Groups vs NACLs, and the full "EC2 application not reachable" checklist are in Chapter 7.

Example: EC2 Website Not Accessible

```
# 1. Linux side: is the service running, listening and answering locally?
sudo systemctl status nginx
sudo ss -tlnp | grep ':80'
curl -I http://localhost

# 2. Host firewall
sudo ufw status                  # Ubuntu
sudo firewall-cmd --list-all     # RHEL-based

# 3. From outside
curl -I http://PUBLIC_IP
```

If it works locally, check the AWS side: public IP or Elastic IP, route table to an Internet Gateway, security group allowing TCP 80, NACL allowing traffic both ways, and load balancer target health if one is used.

> **INTERVIEW LINE**
>
> **A security group allowing port 80 does not guarantee the website works.**
>
> The application must also be running, listening on the right address and port, and reachable through every layer of the network path.

12 AWS Linux Production Scenarios

Scenario 1 **Instance is running but SSH fails**

1. Check the instance status checks in the AWS console.
2. Check the access path: public IP, or bastion / VPN for private instances.
3. Check the security group, NACL and route table.
4. Use Session Manager or the EC2 Serial Console to get in and check the server side:

```
sudo systemctl status ssh        # sshd on Amazon Linux / RHEL
sudo ss -tlnp | grep ':22'
sudo journalctl -u ssh -n 50
```

If no console access works, use the rescue-instance method from Chapter 8.

Scenario 2 EBS volume is attached but not visible

```
lsblk -o NAME,SIZE,SERIAL,MOUNTPOINTS
sudo dmesg -T | tail -20
```

On Nitro instances, a volume attached as `/dev/sdf` appears as `/dev/nvme1n1`. Match the volume ID in the SERIAL column with the AWS console. Confirm the attachment state in AWS before formatting or mounting anything.

Scenario 3 Disk usage keeps increasing

```
df -h; df -i
sudo du -h --max-depth=1 /var | sort -hr | head
sudo lsof +L1
```

Investigate logs, temporary files, database files, local backups and deleted-but-open files. Expand the EBS volume only if the workload genuinely needs more space.

Scenario 4 Application cannot access S3

```
# Which identity is actually being used?
aws sts get-caller-identity

# Can it list the bucket?
aws s3 ls s3://my-bucket/
```

Check that the instance has a role, the role allows the action, the bucket policy does not deny it, KMS key permissions allow encryption or decryption, no old keys override the role, and the region and bucket name are correct. In a private subnet, the instance needs a NAT Gateway or an S3 VPC endpoint.

Scenario 5 Instance is running but the application is down

```
uptime; free -h; df -h
sudo systemctl status myapp
sudo journalctl -u myapp -n 100
```

Then follow the layer-by-layer approach from Chapter 16: process, port, local `curl`, proxy, dependencies and recent deployments.

Scenario 6 **Instance cannot download packages**

```
ip route
getent hosts archive.ubuntu.com
curl -I https://aws.amazon.com
```

A private-subnet instance needs a route to a NAT Gateway. Also check DNS, security group outbound rules, NACLs, the host firewall and any proxy configuration.

13 Key AWS Services for Linux DevOps Engineers

Service	Linux DevOps use
EC2	Run Linux servers
EBS / EFS	Block storage / shared file storage
S3	Backups, artifacts and static files
IAM	Roles and permissions for users and instances
Systems Manager	Session Manager, Run Command, Parameter Store
CloudWatch	Metrics, logs and alarms
VPC	Networking, subnets, routing and security groups
Route 53	DNS
ALB	Distributing HTTP traffic across instances
Auto Scaling	Adding and removing instances automatically
KMS / Secrets Manager	Encryption keys / application secrets

You do not need to master every service at once. Focus on how they fit together in a production architecture.

14 Interview Questions and Answers

Q1. How do you connect to an EC2 Linux instance?

With SSH, using the key pair and the default user, such as `ubuntu` or `ec2-user`. Alternatively, with Session Manager, if the instance has the SSM Agent, an IAM role and a network path to Systems Manager.

Q2. How do you access an EC2 instance without SSH or an open port 22?

With AWS Systems Manager Session Manager. The agent connects outbound to Systems Manager, so no inbound port or SSH key is needed, and access is controlled by IAM.

Q3. What is EC2 User Data?

A script or configuration passed at launch and run by cloud-init, as root, on the first boot. It is used to install packages and configure services automatically.

Q4. You changed the User Data of a running instance. Why did nothing happen?

User Data runs only on the first boot by default. Changing it affects new instances, not one that has already been launched.

Q5. What is IMDSv2, and why is it important?

The token-based version of the instance metadata service. Requiring a token from an HTTP PUT protects against SSRF attacks that try to steal the instance's IAM credentials.

Q6. How do you attach and mount a new EBS volume?

Attach it in AWS, identify it with `lsblk`, create a filesystem only if it is new and empty, mount it, and add a UUID-based `/etc/fstab` entry with `nofail`. Validate with `findmnt --verify` and `mount -a`.

Q7. What is the difference between EBS and EFS?

EBS is block storage normally attached to one instance in one Availability Zone. EFS is shared NFS file storage that many instances across Availability Zones can mount at once.

Q8. Why should you use IAM roles instead of access keys on EC2?

Roles provide temporary credentials that rotate automatically and are managed centrally. Nothing long-lived is stored on the server or in code.

Q9. The instance has a correct IAM role, but the CLI still gets AccessDenied. Why?

Credentials in environment variables or `~/.aws/credentials` take priority over the instance role. Check with `aws sts get-caller-identity`. Also check the bucket policy and KMS key permissions.

Q10. Why is EC2 memory usage not visible in CloudWatch by default, and how do you add it?

Memory is visible only inside the OS. Install the CloudWatch Agent with an IAM role that allows publishing metrics, and load a configuration that collects memory and disk usage.

Q11. How do you troubleshoot an EC2 website that is not accessible?

Check the service, listening port and local `curl` first. If it works locally, check the host firewall, security group, NACL, route table, public IP, DNS and load balancer health.

Q12. A private-subnet instance cannot install packages. What do you check?

Its route table must send internet traffic to a NAT Gateway. Also check DNS, outbound security group rules, NACLs and any proxy settings.

15 Chapter Takeaways

After completing this chapter, you should understand:

- ✓ How Linux EC2 instances are accessed and administered.
- ✓ SSH key pairs and default usernames.
- ✓ Session Manager and its IAM and network requirements.
- ✓ User Data, cloud-init and how to debug them.
- ✓ Instance metadata, IMDSv2 and why it matters.
- ✓ Attaching, mounting and expanding EBS volumes.
- ✓ Mounting shared EFS storage.
- ✓ Using the AWS CLI with IAM roles, and credential precedence.
- ✓ Collecting memory and disk metrics with the CloudWatch Agent.
- ✓ Troubleshooting problems that span Linux and AWS networking.

PRACTICAL EXERCISES

Use a Free Tier or cost-controlled account. Check current pricing first, and delete resources when you finish.

1. Launch an Ubuntu or Amazon Linux instance, connect with SSH, and check the OS and kernel.
2. Install Nginx and open the site using the public IP.
3. Launch an instance with User Data that installs Nginx, then read `/var/log/cloud-init-output.log`.
4. Attach an SSM role and open a Session Manager session without SSH.
5. Retrieve the instance ID and Availability Zone using IMDSv2.
6. Attach a small EBS volume, format it, mount it at `/data`, and configure fstab.
7. Increase the volume size and grow the filesystem.
8. Attach a least-privilege S3 role and upload a file with the CLI, without any access keys.
9. Run `aws sts get-caller-identity` to confirm the role is in use.
10. Install the CloudWatch Agent, publish memory and disk metrics, and create a disk alarm.
11. Block HTTP in a test security group, observe the failure, then restore it.

KEY TAKEAWAY

On AWS, troubleshoot the complete path: the application process and Linux networking, then IAM, security groups, routing and the AWS services the server depends on.

NEXT CHAPTER

Chapter 18: Linux Interview Preparation and Quick Revision

You will revise essential Linux commands, important files and directories, and practise the most frequently asked Linux, AWS, Docker and Kubernetes interview questions.

END OF CHAPTER 17